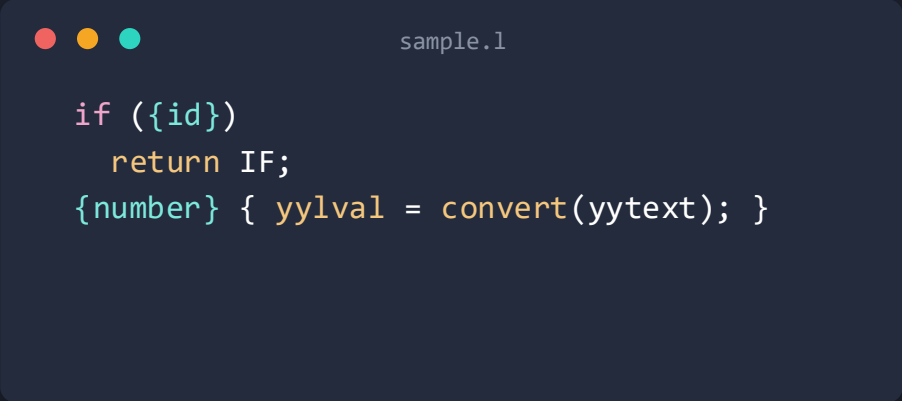




Geradores de Analisadores

Módulo 8 — 8.1 Visão geral · 8.2 Yacc · 8.3 Lex (C) e JFlex (Java)

A code editor window with a dark blue background and three colored window control buttons (red, yellow, green) in the top left corner. The title bar on the right says "sample.l". The code is written in a light blue monospace font.

```
if ({id})  
    return IF;  
{number} { yy1val = convert(yytext); }
```

Agenda

8.1

Visão geral

Como funciona um gerador de compiladores e quais ferramentas existem

8.2

Yacc

Panorama do gerador de analisadores sintáticos (LALR)

8.3

Lex

Gerador de analisadores léxicos: sintaxe, exemplos em C e em Java (JFlex), e boas práticas



Prática

5 exercícios para fixação, do reconhecimento de tokens à tabela de símbolos

8.1

Visão geral

O papel dos geradores de compiladores no processo de tradução

Como um gerador de compiladores trabalha

Um gerador de compiladores recebe uma especificação formal e produz automaticamente o código-fonte de um componente do compilador.



+ classes de usuário (tabela de símbolos, gerador de código, main) → compilados e ligados ao scanner e ao parser → compilador gerado

Exemplos de ferramentas geradoras

Ferramenta	Gera	Linguagens-alvo	Observação
Yacc / Bison	Analisador sintático (LALR)	C, C++ (Bison), Java (via BYacc/J)	Bison é a versão GNU moderna, ainda muito usada
Lex / Flex	Analisador léxico (AFD)	C, C++ (Flex), Java (JFlex), C#	Flex é o sucessor de facto do Lex original
Coco/R	Analisador léxico + sintático	Java, C#, Modula-2, Oberon	Especificação única (léxico + sintático + semântica)
ANTLR	Léxico + sintático + AST	Java, C#, Python, JS, Go...	Ferramenta moderna, gramáticas LL(*), multiplataforma

8.2

Yacc

Panorama rápido — o gerador de analisadores sintáticos

Yacc será detalhado em material próprio. Aqui focamos na integração com o Lex.

- Yet Another Compiler-Compiler (Bell Labs, 1970s)
- Especificação em regras BNF + ações semânticas em C
- Gera `yyparse()`, integrado ao `yylex()` produzido pelo Lex
- Constrói um analisador LALR(1) (bottom-up)

8.3

Lex

Gerador de analisadores léxicos a partir de expressões regulares

História e principais versões

Histórico

1975 — criado nos Laboratórios Bell por Mike Lesk e Eric Schmidt

- Gera um analisador léxico a partir de um Autômato Finito Determinístico (AFD)
- Ferramenta originalmente Unix; hoje disponível também para Windows
- Alvo original: C; versões posteriores geram C++ e Java
- Uso típico: em conjunto com o Yacc, formando o par scanner + parser

Versões atuais

flex *C / C++*

gnu.org/software/flex — versão GNU, padrão de fato hoje em dia

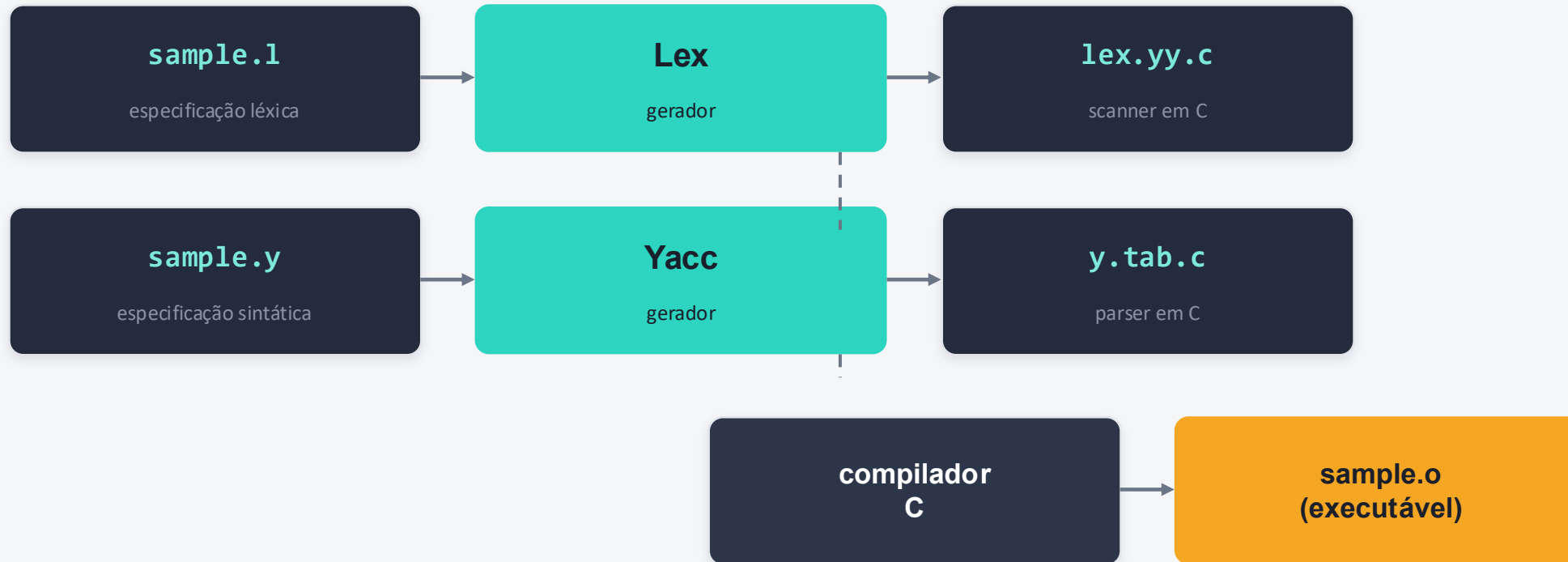
JFlex *Java*

jflex.de — sucessor ativo do JLex (que está descontinuado)

CsLex *C#*

derivado do JLex; pouco mantido — considerar alternativas .NET modernas

Fluxo de compilação com Lex e Yacc



lex.yy.c e y.tab.c são compilados e ligados junto às classes de usuário (tabela de símbolos, gerador de código, main).

Estrutura de uma especificação Lex

1 Declarações

código C + macros de expressões regulares

2 Regras

padrão → ação, separadas por %%

3 Rotinas

funções C auxiliares

```
sample.l

/* declarações: macros de expressões regulares */
delim  [ \t\n]
ws     {delim}+
letter [A-Za-z]
digit  [0-9]
id     {letter}({letter}|{digit})*
number {digit}+
%%

/* regras: padrão { ação } – nomes de token em MAIÚSCULAS */
{ws}    { }
if      { return IF; }
then    { return THEN; }
else    { return ELSE; }
{id}    { yylval.sval = storeId(yytext,yyleng); return ID; }
{number} { yylval.ival = convert(yytext,yyleng); return NUMBER; }
[<>]    { return yytext[0]; }
.       { } /* ignora demais caracteres */
%%

/* rotinas auxiliares em C */
int storeId(char* text, int len) { ... }
```

A mesma especificação em JFlex

JFlex é o gerador ativo para Java (sucessor do JLex). A estrutura em 3 partes é a mesma do Lex — mudam a sintaxe de declaração e a forma de acessar o texto do token.

Lexer.flex

```
%class    Lexer
%type     Token
%function nextToken
%eofval{ return null; %eofval}
%%
/* macros: mesma sintaxe de expressões regulares do Lex */
delim     = [ \t\n]
ws        = {delim}+
letter    = [A-Za-z]
digit     = [0-9]
id        = {letter}({letter}|{digit})*
number    = {digit}+
%%
<YYINITIAL> {
    {ws}      { }
    "if"      { return new Token(TokenType.IF); }
    "then"    { return new Token(TokenType.THEN); }
    {id}      { return new Token(TokenType.ID, yytext()); }
    {number}  { return new Token(TokenType.NUMBER,
                          Integer.parseInt(yytext())); }
    .        { }
}
```

Correções em relação à especificação clássica



NUMBER, não "number"

Nomes de token devem ser consistentes em MAIÚSCULAS, como IF/THEN/ELSE — evita conflito com identificadores em C e mantém o padrão esperado pelo Yacc.



yyval como union/struct

Na prática (Bison/Yacc real), yyval é do tipo YYSTYPE — normalmente uma union — pois pode carregar valores de tipos diferentes (int, string, nó de AST), não apenas int.



Regra do ponto (.)

"." casa qualquer caractere exceto \n; deve vir por último nas regras para funcionar como 'catch-all', nunca antes das regras específicas.

O analisador léxico gerado

```
scanner (lex.yy.c)

int yylex() { ... }

int yylval; // valor semântico (ou YYSTYPE)
char* yytext; // texto do token reconhecido
int yyleng; // tamanho do texto do token
int yylineno; // linha atual no arquivo-fonte
```

- A especificação léxica é traduzida em uma função `int yylex()`, chamada pelo parser a cada novo token.
- `yylex()` devolve o código do token e expõe atributos via variáveis globais.
- O parser (Yacc) declara e exporta os códigos de token:

```
sample.y

%token IF
%token THEN
%token ELSE
%token ID NUMBER
```

Lex (C) vs JFlex (Java): principais diferenças

Aspecto	Lex (C)	JFlex (Java)
Extensão do arquivo	<code>.l</code>	<code>.flex</code> / <code>.jflex</code>
Bloco de declarações	<code>%{ código C %}</code>	<code>%class, %type, %function, import</code>
Texto do token	<code>ytext</code> (variável global <code>char*</code>)	<code>ytext()</code> (método da classe gerada)
Tamanho do token	<code>yleng</code> (variável)	<code>ylength()</code> (método)
Linha atual	<code>yylineno</code> (variável)	<code>yyline()</code> (método, conta a partir de 0)
Retorno da ação	<code>int</code> (código exportado pelo Yacc)	qualquer tipo Java – normalmente um objeto <code>Token</code>
Integração com parser	Yacc / Bison (<code>yyparse</code> chama <code>yylex</code>)	<code>java_cup</code> ou ANTLR (ou parser manual)

Expressões regulares no Lex

abc	a string literal "abc"; cada caractere, exceto () [] { } * + ? ^ \$. \, denota a si próprio
.	qualquer caractere, exceto \n (fim de linha)
x*	0 ou mais repetições de x
x+	1 ou mais repetições de x
x?	0 ou 1 ocorrência de x (opcional)
(x y)	agrupamento e alternativa entre x e y
[...]	conjunto de caracteres entre colchetes, ex.: [A-Za-z0-9\$]
{...}	uso de uma macro previamente definida
^ \$	início de linha / fim de linha (^ dentro de [] indica negação)
\uNNNN	caractere Unicode pelo código hexadecimal

Duas convenções essenciais do Lex

1

Casamento mais longo (maximal munch)

O analisador léxico sempre reconhece o token com a maior sequência possível de caracteres.

```
entrada: "iff"  
resultado: ID ("iff") – não IF + f
```

Outro exemplo: "==" deve ser reconhecido inteiro como operador de igualdade, e não como dois "=" separados.

2

Prioridade por ordem de declaração

Em caso de empate no tamanho do casamento, vence a regra declarada primeiro no arquivo .l.

```
if    { return IF; }    // 1ª: prioridade  
{id} { return ID; }    // 2ª: identificadores  
  
entrada: "if"  
resultado: IF (não ID, mesmo "if" casar com {id})
```


Especificação Lex para uma calculadora simples

Reconhece números, operadores aritméticos, parênteses e ignora espaços — pronta para alimentar um parser Yacc de expressões.

```
calc.l

%{
#include "y.tab.h"  /* códigos de token gerados pelo Yacc */
%}

digit    [0-9]
number   {digit}+(\.{digit}+)?
%%

[ \t]+   { }           /* ignora espaços/tabs */
\n       { return '\n'; } /* fim de expressão */
{number} { yylval.dval = atof(yytext); return NUMBER; }
"+"|"-"|"*"|"/" { return yytext[0]; } /* operadores */
"("|")"   { return yytext[0]; } /* parênteses */
.         { fprintf(stderr, "erro léxico: %s\n", yytext); }
%%
```

A mesma calculadora em JFlex

Em Java, o Lexer gerado pelo JFlex é tipicamente pareado com o CUP (Constructor of Useful Parsers), o "Yacc" do mundo Java.

CalcLexer.flex

```
import java_cup.runtime.Symbol;
%%
%class    CalcLexer
%type     Symbol
%eofval{ return symbol(sym.EOF); %eofval}
%%
digit     = [0-9]
number    = {digit}+(\.{digit}+)?
%%
<YYINITIAL> {
    [ \t\r]+ { } /* ignora espaços/tabs */
    \n      { } /* fim de linha */
    {number} { return symbol(sym.NUMBER,
                          Double.parseDouble(yytext())); }
    "+"     { return symbol(sym.PLUS); }
    "-"     { return symbol(sym.MINUS); }
    "*"     { return symbol(sym.TIMES); }
    "/"     { return symbol(sym.DIV); }
    "."     { System.err.println("erro léxico: " + yytext()); }
}
```

Lex no ecossistema atual

Lex/Flex

+ Rápido, maduro, gera C/C++ direto

– Sintaxe antiga; só léxico (precisa de Yacc separado)

JFlex

+ Ativo, bem documentado para Java

– Ecossistema menor que o do ANTLR

ANTLR

+ Léxico + sintático + AST integrados; multiplataforma

– Curva de aprendizado maior; runtime próprio

PLY (Python)

+ Muito simples de prototipar

– Desempenho inferior às ferramentas em C

Pontos-chave do módulo

1 Geradores de compiladores automatizam a construção de scanners e parsers a partir de especificações formais.

2 O Lex traduz expressões regulares em um AFD e gera a função `yylex()`, usada pelo parser.

3 Duas regras governam o casamento de padrões: maior sequência possível (maximal munch) e prioridade por ordem de declaração.

4 Na prática, Lex é usado em conjunto com o Yacc — hoje, com flex/Bison ou ferramentas integradas como ANTLR.

Exercícios propostos

1

Analizador de expressões aritméticas

Em C (flex) ou Java (JFlex), reconheça NUM, PLUS, MINUS, TIMES, DIV, LPAREN, RPAREN a partir de expressões como $(3 + 4) * 2$. Teste com pelo menos 5 entradas.

2

Comentários de linha e de bloco

Adicione regras para ignorar `//` até o fim da linha e `/* ... */` (sem aninhamento). Trate comentário de bloco não fechado como erro léxico, reportando `yylineno`.

3

Estatísticas do código-fonte

Usando ações Lex, conte o número de identificadores, números e palavras reservadas (`if/then/else/while`) em um arquivo de entrada e imprima o resumo ao final.

4

Deteccão de erros léxicos

Trate strings não fechadas (`"texto sem aspas final`) e caracteres inválidos (ex.: `@`, `#`), emitindo mensagens de erro com número da linha (`yylineno`).

5

Tabela de símbolos simples

Implemente `storeId()` para armazenar identificadores distintos em uma tabela (vetor ou lista ligada) e imprima a tabela completa ao final da análise.

Bibliografia deste módulo

Referência principal do curso

AHO, A. V.; SETHI, S.; ULLMAN, J. D. Compilers: Principles, Techniques, and Tools (também disponível em português: Compiladores: princípios, técnicas e ferramentas, 2ª ed., Pearson, 2008) — Cap. 3, Análise Léxica.

- LEVINE, J. R.; MASON, T.; BROWN, D. Lex & Yacc. O'Reilly, 1998.
- APPEL, A. W. Modern Compiler Implementation in Java, 2ª ed. Cambridge, 2002. (base para os exemplos em Java)
- Documentação oficial — C: gnu.org/software/flex · Java: jflex.de (manual) e projeto CUP (java_cup)
- Material original adaptado: dotnet.jku.at/courses/CC/ (versão reduzida, revisada e ampliada nesta apresentação)